

Unity extOSC

실전 프로젝트 강의

Hello OSC부터 URP 렌더 제어, 운영자 패널, 프리셋 Bundle, 디버그 대시보드까지 직접 만드는 수업용 자료입니다.



Send



Receive



Message



Bundle

이론 먼저가 아님

각 장은 바로 Unity에서 클릭하거나 코드를 붙이는 실습으로 시작합니다.

결과물이 남음

각 Lab 끝에는 캡처, 주소표, README 문장, 테스트 표 같은 제출물이 생깁니다.

포트폴리오 관점

단순 송수신이 아니라 렌더/카메라/조명/운영 UI를 연결합니다.

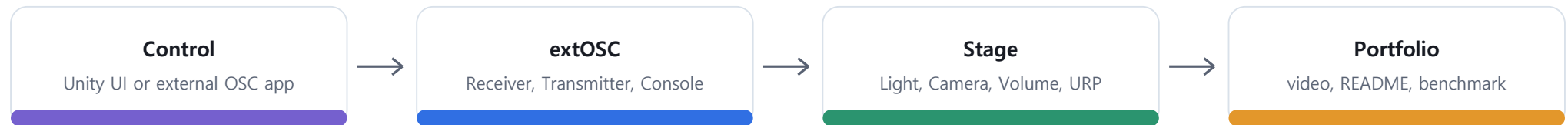
수업 진행 방식

페이지마다 '목표 -> 만들기 -> 코드 -> 확인 -> 흔한 실수 -> 포트폴리오 기록' 순서로 봅니다. 모르는 개념은 실습이 끝난 뒤 되돌아와 확인하는 방식이 더 빠릅니다.

One packet, one job



이 수업의 목표는 '외부 OSC 신호로 Unity 라이브 스테이지를 제어하는 작은 운영 시스템'을 만드는 것입니다. 완성 후에는 데모 영상, README, 주소 계약서, 성능 비교 표까지 포트폴리오로 남깁니다.



썸 1

OSC_Basics: Hello 메시지
송수신과 Console 확인

썸 2

LiveStage:
조명/카메라/이펙트/프리셋 제어

썸 3

RenderLab: renderScale과 Bloom
값을 OSC로 조절

썸 4

DebugPanel: Ping, last message,
error count 표시

Scene hierarchy blueprint

GameObject	Attach / purpose
OSC_System	OSCReceiver, OSCTransmitter, DebugHUD
Stage_Root	Light, Camera, Volume, props
ControlPanel_Canvas	Slider, Button, Pad, status text
Controllers	LightController, CameraController, URPController, PresetController

실습 전 체크리스트

1. Package Manager에서 extOSC가 보이는지 확인
2. Samples 중 Getting Started, Scripting, UI, Mapping, Ping을 Import
3. Tools > extOSC > OSC Console 메뉴가 보이는지 확인
4. 빈 Scene을 하나 만들고 OSC_System 오브젝트를 생성
5. 테스트 포트는 우선 7001 하나로 통일

비유

실습 전 준비는 촬영장 세팅입니다. 카메라, 조명, 무전기, 콘솔 위치가 잡혀야 감독의 큐 사인이 의미가 생깁니다.

아래 순서대로 진행하면 각 실습이 다음 실습의 기반이 됩니다. 막히면 이전 Lab의 통과 기준으로 돌아가세요.

Lab	만드는 것	통과 기준	포트폴리오 증거
0	설치/샘플/씬 구조	extOSC Console과 샘플 확인	환경 캡처
1	Hello OSC Loopback	문자열 메시지 송수신	Console 로그
2	주소 계약서	주소/타입/범위 표 완성	README 표
3	Light Controller	OSC slider로 조명 변화	GIF/영상
4	Camera Controller	FOV/위치 오프셋 제어	연출 전환 영상
5	URP Render Controller	renderScale/Bloom 값 변경	Profiler 비교
6	Preset Bundle	한 명령으로 무대 상태 전환	시연 영상
7	Debug Dashboard	Ping/마지막 메시지 표시	운영 UI 캡처

목표: extOSC가 설치되어 있다는 사실을 실습 환경으로 바꾸기

패키지가 설치된 것과 수업을 시작할 수 있는 것은 다릅니다. Lab 0에서는 설치 확인, 샘플 Import, 실습 씬 구조, 포트 규칙을 한 번에 정리합니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

환경 캡처 3장: Package Manager extOSC 화면,
Tools > extOSC 메뉴, OSC_System hierarchy

Unity 안에서 그대로 따라 하는 체크리스트입니다.

순서	Unity에서 할 일	확인할 것
1	Window > Package Manager 열기	extOSC 패키지 표시
2	Samples에서 Getting Started, Scripting, UI, Mapping, Ping Import	Assets/Samples 또는 Imported sample 확인
3	Tools > extOSC > OSC Console 열기	빈 콘솔 창 표시
4	새 Scene 생성: OSC_Practical_Lab	Hierarchy에 OSC_System 생성
5	OSC_System에 Receiver, Transmitter 추가	Receiver LocalPort 7001, Transmitter RemotePort 7001
6	Scene 저장	Assets/Scenes/OSC_Practical_Lab.unity

목표: 같은 컴퓨터 안에서 OSC 메시지가 실제로 왕복하는지 확인

localhost `127.0.0.1`과 port `7001`만 써서 가장 작은 OSC 송수신을 만듭니다. 이 실습이 통과돼야 이후 조명/카메라/렌더 제어가 의미를 가집니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

OSC Console 로그와 Unity Console 로그

버튼 없이 Play 시작 시 한 번 보내는 코드입니다. extOSC README의 기본 송신 구조를 실습용 주소로 바꾼 형태입니다.

C# example

```
using extOSC;
using UnityEngine;

public class Lab01SendHello : MonoBehaviour
{
    public OSCTransmitter transmitter;

    private void Start()
    {
        var message = new OSCMessage("/lab/hello");
        message.AddValue(OSCValue.String("Hello extOSC"));
        transmitter.Send(message);
    }
}
```

읽는 법

- `/lab/hello`는 이 수업에서 첫 번째로 약속하는 주소입니다.
- Transmitter의 RemoteHost는 `127.0.0.1`, RemotePort는 `7001`로 둡니다.
- 외부 앱 없이 자기 자신에게 보내는 loopback 구조라 네트워크 변수를 최소화합니다.

Receiver는 주소와 콜백을 묶습니다. 메시지가 들어오면 주소 비교 후 콜백이 실행됩니다.

C# example

```
using extOSC;
using UnityEngine;

public class Lab01ReceiveHello : MonoBehaviour
{
    public OSCReceiver receiver;

    private void Start()
    {
        receiver.Bind("/lab/hello", OnHello);
    }

    private void OnHello(OSCMessage message)
    {
        if (message.ToString(out var text))
            Debug.Log($"OSC Hello: {text}");
    }
}
```

읽는 법

- Receiver의 LocalPort는 `7001`입니다.
- `message.ToString(out var text)`는 extOSC 확장 메서드입니다.
- Unity Console에 `OSC Hello: Hello`
extOSC`가 보이면 통과입니다.

Hello OSC에서 자주 막히는 부분을 먼저 잡아두면 이후 실습에서 시간을 많이 아낍니다.

증상	가장 먼저 볼 것	해결
아무 로그도 없음	Receiver LocalPort와 Transmitter RemotePort	둘 다 7001인지 확인
Socket Error	다른 앱이 같은 포트를 쓰는지	포트 변경 또는 해당 앱 종료
OSC Console에는 보임, Unity Console에는 없음	Bind 주소 오타	`/lab/hello` 완전 일치 확인
값이 안 읽힘	타입 불일치	송신 String, 수신 ToString인지 확인

목표: 코드보다 먼저 OSC 주소표를 설계하기

실전에서는 주소가 API입니다. 주소표 없이 코드를 쓰면 나중에 UI, 외부 앱, README, 테스트가 서로 다른 말을 하게 됩니다.

1. 만들기

썬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

README에 붙일 Address Contract Table

첫 버전은 아래 정도면 충분합니다. 모든 주소는 나중에 외부 패널이나 테스트 앱에서 그대로 사용할 수 있어야 합니다.

Address	Type	Range	Unity target
/stage/light/key/intensity	float	0-8	Key Light.intensity
/stage/light/key/color	float,float,float	0-1 RGB	Key Light.color
/camera/main/fov	float	25-70	Camera.fieldOfView
/camera/main/offset	float,float	-3 to 3	Camera rig local position
/render/scale	float	0.5-1.0	URP Asset renderScale
/render/bloom/intensity	float	0-10	VolumeProfile Bloom.intensity
/stage/preset/load	string	preset id	Preset controller
/system/ping	impulse	none	Connection monitor

주소 문자열은 여기저기 직접 쓰지 말고 한 곳에 모읍니다. 오타를 줄이고 README 표와 코드가 같은 이름을 쓰게 만드는 작은 습관입니다.

C# example

```
public static class OscAddress
{
    public const string KeyLightIntensity = "/stage/light/key/intensity";
    public const string KeyLightColor = "/stage/light/key/color";
    public const string CameraFov = "/camera/main/fov";
    public const string CameraOffset = "/camera/main/offset";
    public const string RenderScale = "/render/scale";
    public const string BloomIntensity = "/render/bloom/intensity";
    public const string StagePresetLoad = "/stage/preset/load";
    public const string SystemPing = "/system/ping";
}
```

읽는 법

- 이 파일은 `Assets/Scripts/OscAddress.cs`로 두면 됩니다.
- 주소를 바꿀 때 코드 전체 검색이 아니라 이 파일과 README 표만 보면 됩니다.
- 마스크 주소를 쓰기 전에는 정확한 주소로 먼저 통과시키세요.

목표: OSC float 값 하나로 실제 렌더 결과를 바꾸기

조명은 가장 즉각적으로 보이는 실습 대상입니다. `/stage/light/key/intensity`로 Light intensity를 바꾸고, 색상까지 확장합니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

조명 밝기 Before/After 캡처와 10초 데모 영상

float 하나를 받아 Light intensity에 반영합니다. 외부 입력은 항상 clamp해서 안전한 범위 안에 넣습니다.

C# example

```
using extOSC;
using UnityEngine;

public class Lab03LightController : MonoBehaviour
{
    public OSCReceiver receiver;
    public Light keyLight;

    private void Start()
    {
        receiver.Bind(OscAddress.KeyLightIntensity, OnIntensity);
        receiver.Bind(OscAddress.KeyLightColor, OnColor);
    }

    private void OnIntensity(OSCMessage message)
    {
        if (!message.ToFloat(out var value)) return;
        keyLight.intensity = Mathf.Clamp(value, 0f, 8f);
    }

    private void OnColor(OSCMessage message)
    {
        if (message.Values.Count < 3) return;
        var r = Mathf.Clamp01(message.Values[0].FloatValue);
        var g = Mathf.Clamp01(message.Values[1].FloatValue);
        var b = Mathf.Clamp01(message.Values[2].FloatValue);
        keyLight.color = new Color(r, g, b);
    }
}
```

읽는 법

- 실전에서는 intensity와 color를 분리해 주소를 설계합니다.
- 색상은 float 3개를 RGB로 해석합니다. 타입 검사 코드를 더 엄격히 붙이면 더 좋습니다.
- 조명 변화는 Game View에서 바로 확인되므로 첫 포트폴리오 데모에 적합합니다.

조명 제어는 감각으로만 보지 말고 테스트 값과 기대 결과를 표로 남깁니다.

보낼 메시지	기대 결과	캡처
/stage/light/key/intensity 0	Key Light가 꺼진 것처럼 어두워짐	Before
/stage/light/key/intensity 4	중간 밝기	Mid
/stage/light/key/intensity 8	최대 밝기	After
/stage/light/key/color 1 0.6 0.3	따뜻한 색 조명	Color A
/stage/light/key/color 0.4 0.7 1	차가운 색 조명	Color B

목표: OSC 값으로 방송 연출 파라미터를 바꾸기

라이브 콘텐츠 개발자는 렌더 품질뿐 아니라 장면 전환과 카메라 연출도 다룹니다. FOV와 오프셋을 OSC로 조절합니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

FOV 변화 영상과 카메라 오프셋 테스트 캡처

기본 Camera만으로도 실습할 수 있게 구성했습니다. Cinemachine을 쓰는 경우에는 같은 값을 Virtual Camera 쪽으로 연결하면 됩니다.

C# example

```
using extOSC;
using UnityEngine;

public class Lab04CameraController : MonoBehaviour
{
    public OSCReceiver receiver;
    public Camera targetCamera;
    public Transform cameraRig;

    private void Start()
    {
        receiver.Bind(OscAddress.CameraFov, OnFov);
        receiver.Bind(OscAddress.CameraOffset, OnOffset);
    }

    private void OnFov(OSCMessage message)
    {
        if (!message.ToFloat(out var value)) return;
        targetCamera.fieldOfView = Mathf.Clamp(value, 25f, 70f);
    }

    private void OnOffset(OSCMessage message)
    {
        if (message.Values.Count < 2) return;
        var x = Mathf.Clamp(message.Values[0].FloatValue, -3f, 3f);
        var y = Mathf.Clamp(message.Values[1].FloatValue, -2f, 2f);
        cameraRig.localPosition = new Vector3(x, y, cameraRig.localPosition.z);
    }
}
```

읽는 법

- Camera와 cameraRig를 분리하면 회전/위치/줌 역할을 나누기 쉽습니다.
- 오프셋은 Pad UI와 궁합이 좋습니다.
- 갑작스러운 점프가 싫다면 이후 단계에서 Mathf.Lerp 또는 SmoothDamp를 추가합니다.

외부 OSC 값은 즉시 바꾸는 것보다 target 값을 저장하고 Update에서 보간하면 방송 화면이 덜 거칠어집니다.

C# example

```
public class SmoothFloat
{
    public float current;
    public float target;
    public float speed = 8f;

    public float Tick(float deltaTime)
    {
        current = Mathf.Lerp(current, target, 1f - Mathf.Exp(-speed * deltaTime));
        return current;
    }
}

// Usage idea:
// OnFov message: fov.target = Mathf.Clamp(value, 25f, 70f);
// Update: targetCamera.fieldOfView = fov.Tick(Time.deltaTime);
```

읽는 법

- OSC는 값 전달, 보간은 Unity 쪽 연출 책임으로 나눕니다.
- 빠른 반응이 중요한 버튼은 즉시 적용하고, 카메라/조명은 보간하는 편이 자연스럽습니다.
- 포트폴리오 설명에서 '운영 입력과 화면 적용을 분리했다'고 말할 수 있습니다.

목표: OSC로 렌더 품질과 비용을 비교하는 실험 패널 만들기

URP renderScale과 Bloom intensity를 OSC로 바꾸고, Profiler에서 프레임 비용 변화를 기록합니다. 렌더 최적화 포트폴리오와 가장 직접적으로 연결되는 실습입니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler 중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

renderScale 1.0/0.8/0.67 비교 표와 Game View 캡처

로컬 URP 17.3.0 소스에서 `UniversalRenderPipelineAsset.renderScale`은 get/set 가능한 프로퍼티로 확인됩니다. 실습에서는 별도 URP Asset을 참조해 값을 바꿉니다.

C# example

```
using extOSC;
using UnityEngine;
using UnityEngine.Rendering.Universal;

public class Lab05RenderScaleController : MonoBehaviour
{
    public OSCReceiver receiver;
    public UniversalRenderPipelineAsset urpAsset;

    private void Start()
    {
        receiver.Bind(OscAddress.RenderScale, OnRenderScale);
    }

    private void OnRenderScale(OSCMessage message)
    {
        if (urpAsset == null) return;
        if (!message.ToFloat(out var value)) return;

        urpAsset.renderScale = Mathf.Clamp(value, 0.5f, 1.0f);
    }
}
```

읽는 법

- 주의: 프로젝트 공용 URP Asset을 런타임에서 바꾸면 전체 카메라 렌더링에 영향을 줄 수 있습니다.
- 포트폴리오 실험용 URP Asset을 따로 복제해 사용하는 편이 안전합니다.
- renderScale은 품질/성능 비교용으로 좋지만 UI는 네이티브 해상도로 렌더된다는 Unity UI 설명도 함께 기억하세요.

URP Bloom은 VolumeProfile 안의 VolumeComponent입니다. 로컬 URP 소스의 예제도 VolumeProfile에서 component를 가져와 runtime에 값을 바꾸는 흐름을 보여줍니다.

C# example

```
using extOSC;
using UnityEngine;
using UnityEngine.Rendering;
using UnityEngine.Rendering.Universal;

public class Lab05BloomController : MonoBehaviour
{
    public OSCReceiver receiver;
    public VolumeProfile volumeProfile;
    private Bloom bloom;

    private void Start()
    {
        if (volumeProfile != null)
            volumeProfile.TryGet(out bloom);

        receiver.Bind(OscAddress.BloomIntensity, OnBloom);
    }

    private void OnBloom(OSCMessage message)
    {
        if (bloom == null) return;
        if (!message.ToFloat(out var value)) return;

        bloom.intensity.overrideState = true;
        bloom.intensity.value = Mathf.Clamp(value, 0f, 10f);
    }
}
```

읽는 법

- VolumeProfile에 Bloom override가 없으면 `TryGet`은 false가 됩니다.
- 실습 전 Global Volume에 Bloom을 추가하고 intensity Override 체크를 켭니다.
- 런타임 수정용 Profile은 원본을 복제해 쓰면 에셋 오염을 줄일 수 있습니다.

수치는 네 노트북/씬 기준으로 직접 채워 넣는 칸입니다. 중요한 것은 '설정 변경 -> 화면 변화 -> 비용 변화'를 같은 표에 묶는 것입니다.

Test	Message	Visual result	CPU/GPU ms	Note
A	/render/scale 1.0	Native render target		baseline
B	/render/scale 0.8	약간 부드러워짐		quality/perf compare
C	/render/scale 0.67	픽셀 확대감 증가		low scale test
D	/render/bloom/intensity 0	Bloom off		post cost compare
E	/render/bloom/intensity 5	Bloom visible		post cost compare

목표: 한 번의 명령으로 여러 무대 상태를 바꾸기

실제 운영에서는 조명 하나만 바꾸기보다 '무대 A', '토크 모드', '클라이맥스 효과'처럼 여러 값을 묶어 전환합니다. 이때 주소 계약과 Bundle 개념이 같이 필요합니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

프리셋 3개와 전환 영상

`/stage/preset/load`로 문자열 id를 받아 여러 컨트롤러에 값을 넘기는 구조입니다. 직접 값을 바꾸지 말고 전용 controller에 위임하면 유지보수가 쉽습니다.

C# example

```
using extOSC;
using UnityEngine;

[System.Serializable]
public class StagePreset
{
    public string id;
    public float keyIntensity;
    public float cameraFov;
    public float renderScale;
}

public class Lab06PresetReceiver : MonoBehaviour
{
    public OSCReceiver receiver;
    public StagePreset[] presets;
    public Lab03LightController lightController;
    public Lab04CameraController cameraController;
    public Lab05RenderScaleController renderController;

    private void Start()
    {
        receiver.Bind(OscAddress.StagePresetLoad, OnLoadPreset);
    }

    private void OnLoadPreset(OSCMessage message)
    {
        if (!message.ToString(out var id)) return;

        foreach (var preset in presets)
        {
            if (preset.id != id) continue;
            Debug.Log($"Load preset: {id}");
            // In production, call explicit Apply methods on each controller.
            return;
        }
    }
}
```

읽는 법

- 여기서는 구조를 보여주기 위해 Apply 호출부를 주석으로 남겼습니다.
- 실제 구현에서는 각 Controller에 `ApplyIntensity`, `ApplyFov`, `ApplyRenderScale` 메서드를 만드세요.
- 프리셋 데이터는 ScriptableObject로 옮기면 포트폴리오 완성도가 올라갑니다.

여러 메시지를 한 타이밍에 보내고 싶을 때 Bundle을 씁니다. OSC 스펙은 Packet 내용이 Message 또는 Bundle이라고 정의합니다.

C# example

```
using extOSC;
using UnityEngine;

public class Lab06PresetSender : MonoBehaviour
{
    public OSCTransmitter transmitter;

    public void SendTalkPreset()
    {
        var bundle = new OSCBundle();
        bundle.AddPacket(OSCMessage.Create(
            OscAddress.KeyLightIntensity, OSCValue.Float(3.0f)));
        bundle.AddPacket(OSCMessage.Create(
            OscAddress.CameraFov, OSCValue.Float(42f)));
        bundle.AddPacket(OSCMessage.Create(
            OscAddress.RenderScale, OSCValue.Float(0.8f)));

        transmitter.Send(bundle);
    }
}
```

읽는 법

- Bundle은 '여러 메시지를 한 봉투에 담기'입니다.
- 수신 쪽이 Bundle을 풀어 각 메시지를 처리하는 흐름은 extOSC Receiver 내부 처리와도 연결됩니다.
- 한 프레임에 많은 값을 보내는 경우 UseBundle과 빈도 제한을 같이 고려합니다.

목표: 만든 기능이 방송 중에도 상태를 설명하게 만들기

실무형 도구는 동작만 하면 끝이 아닙니다. 마지막 메시지, 연결 상태, 현재 프리셋, 경고를 표시해야 운영자가 믿고 쓸 수 있습니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

DebugPanel Game View 캡처

주소 마스크 ``\*``를 사용해 모든 메시지를 받아 UI 텍스트에 마지막 메시지 정보를 남깁니다.

C# example

```
using extOSC;
using UnityEngine;
using UnityEngine.UI;

public class Lab070scDebugHud : MonoBehaviour
{
    public OSCReceiver receiver;
    public Text lastMessageText;
    public int receivedCount;

    private void Start()
    {
        receiver.Bind("*", OnAnyMessage);
    }

    private void OnAnyMessage(OSCMessage message)
    {
        receivedCount++;
        lastMessageText.text =
            $"#{receivedCount} {message.Address} values:{message.Values.Count}";
    }
}
```

읽는 법

- ``\*`` 마스크는 모든 주소를 받으므로 디버그 전용으로 쓰는 편이 안전합니다.
- 운영 UI에는 마지막 주소, 값 개수, 시간, 연결 상태를 표시하세요.
- UI Text 대신 TMP를 써도 구조는 같습니다.

이 항목을 넣으면 단순 예제에서 운영 도구로 한 단계 올라갑니다.

항목	표시 내용	왜 필요한가
Connection	Ping OK / Timeout	외부 앱 연결 상태 파악
Last Address	마지막 수신 OSC 주소	주소 오타 추적
Last Value	값 개수와 대표 값	타입 불일치 추적
Current Preset	현재 무대 프리셋 id	운영 상태 공유
Message Rate	최근 1초 수신 개수	메시지 폭주 감지
Warnings	type mismatch, missing component	실패 원인 표시

목표: OSC 기반 아바타 제어를 단정하지 않고 안전하게 설계하기

VMC Protocol은 OSC over UDP/IP를 쓰지만, extOSC는 VMC 전용 라이브러리가 아니라 일반 OSC 송수신 도구입니다. 따라서 주소와 타입 규약을 별도로 맞춰야 합니다.

1. 만들기

씬/오브젝트/스크립트를 직접 만듭니다.

2. 확인

OSC Console, Unity Console, Game View, Profiler
중 필요한 도구로 통과 기준을 봅니다.

3. 남기기

VMC 수신 로그 캡처와 해석 표

실제 VRM 런타임 API에 붙이기 전, Animator 파라미터로 표정 weight 라우팅 구조를 먼저 검증합니다.

C# example

```
using extOSC;
using UnityEngine;

public class Lab08ExpressionRouter : MonoBehaviour
{
    public OSCReceiver receiver;
    public Animator animator;

    private void Start()
    {
        receiver.Bind("/avatar/expression/*/weight", OnExpression);
    }

    private void OnExpression(OSCMessage message)
    {
        if (!message.ToFloat(out var weight)) return;

        var parts = message.Address.Split('/');
        if (parts.Length < 4) return;

        var expression = parts[3];
        animator.SetFloat(expression, Mathf.Clamp01(weight));
    }
}
```

읽는 법

- 이 코드는 VMC 구현이 아니라 주소 라우팅 연습입니다.
- 실제 VRM Expression 적용은 사용하는 VRM 런타임 API에 맞게 바뀌어야 합니다.
- 마스크 주소를 쓸 때는 허용 expression 목록을 두고 검증하면 더 안전합니다.

보내는 빈도 제한

슬라이더를 매 프레임 보내기보다 20-30Hz 또는 값 변경 시 전송을 우선합니다.

값 검증

타입, 범위, 대상 컴포넌트 null 여부를 검사하고 실패 원인을 HUD에 남깁니다.

Bundle 사용

동시에 바뀌는 상태는 Bundle 또는 프리셋 명령으로 묶어 관리합니다.

Drown Detection을 보는 관점

extOSC changelog와 Receiver 소스에는 수신 처리 과부하를 감지하는 흐름이 있습니다. 경고가 보이면 OSC가 잘못됐다는 뜻보다 'Unity 메인 스레드에서 처리할 양을 줄여야 한다'는 신호로 봅니다.

실습 중 막히면 아래 순서로만 확인하세요. 이 순서가 제일 빠릅니다.

단계	질문	도구
1	패키지가 설치됐나?	Package Manager, manifest.json
2	송신 포트와 수신 포트가 같은가?	Transmitter/Receiver Inspector
3	주소가 완전히 같은가?	OSC Console, Address Contract
4	값 타입이 맞는가?	ToFloat/ToString, Match Pattern
5	Unity 대상이 null이 아닌가?	Inspector reference, Debug.LogWarning
6	메시지가 너무 많은가?	Profiler, Message Rate HUD

하루에 하나씩 끝내면 결과물이 남는 일정입니다. 깊게 파는 것보다 매일 통과 기준을 남기는 쪽이 좋습니다.

Day	작업	끝났다는 기준
1	Lab 0-1: 설치, 샘플, Hello OSC	Hello 로그 캡처
2	Lab 2: 주소 계약서와 OscAddress.cs	README 주소표 1차
3	Lab 3: 조명 제어	밝기/색상 영상
4	Lab 4: 카메라 제어	FOV/오프셋 영상
5	Lab 5: URP 렌더 제어	Profiler 비교 표
6	Lab 6-7: 프리셋과 디버그 패널	운영 UI 캡처
7	문서/영상 정리	1분 데모 영상과 README 완성

이 표를 README 마지막에 그대로 넣고 체크해도 됩니다.

구분	필수 산출물	왜 중요한가
Demo	1분 데모 영상	기능을 한 번에 보여줌
Docs	OSC Address Contract	협업 가능한 설계 증명
Code	Controller scripts	구현 역량 증명
Debug	OSC Console/HUD 캡처	문제 해결 역량 증명
Performance	renderScale/Bloom 비교 표	렌더 최적화 연결
Caution	VMC/OBS는 제안 구조로 명시	근거와 추정을 구분

이번 실전 강의의 핵심은 extOSC 자체를 많이 아는 것이 아니라, OSC 메시지를 Unity 안의 실제 렌더/연출 시스템으로 안전하게 연결하는 것입니다.

1. 주소는 API

주소표를 먼저 만들고 모든 코드와 UI가 그 표를 따르게 합니다.

2. 값은 검증

타입, 범위, null, 실패 로그를 기본으로 넣습니다.

3. 결과는 측정

렌더 스케일과 Bloom은 화면 캡처와 Profiler 수치를 함께 남깁니다.

면접에서 한 문장으로 말하기

extOSC를 이용해 OSC 주소 계약서를 설계하고, 외부 제어 신호를 Unity의 조명/카메라/URP 렌더 파라미터에 연결했으며, Debug HUD와 Profiler 측정으로 운영성과 성능을 함께 검증했습니다.

이 자료는 공식 문서와 현재 프로젝트의 로컬 패키지 파일을 근거로 작성했습니다. 실습 구조와 포트폴리오 과제는 사용자의 목표에 맞춘 설계 제안이며, 특정 기업의 실제 내부 기술 스택을 단정하지 않습니다.

확인된 사실

OSC 스펙, extOSC README/manifest/source, Unity Package Manager/Samples 문서, VMC/OBS 공식 안내, 로컬 URP API

실습 설계

조명/카메라/URP/프리셋/디버그 패널을 연결하는 포트폴리오 구성은 이 프로젝트 목표에 맞춘 수업 설계입니다.

로컬 기준

이 프로젝트의 extOSC는 Git UPM dependency로 설치되어 있고, lock 파일 기준 hash는 b7c2bfa81633cbcbc8cc4312e15cb5fbd0ed7d1d입니다.

S1

OpenSoundControl Specification 1.0

OpenSoundControl.org / Matt Wright

https://opensoundcontrol.stanford.edu/spec-1_0.html

S2

extOSC GitHub README

Iam1337 / dr. ext

<https://github.com/Iam1337/extOSC>

S3

OpenUPM extOSC package page

OpenUPM

<https://openupm.com/packages/com.iam1337.extosc/>

S4

Unity Manual - Install a UPM package from a Git URL

Unity Technologies

<https://docs.unity3d.com/Manual/upm-ui-giturl.html>

S5

Unity Manual - Create samples for your package

Unity Technologies

<https://docs.unity3d.com/Manual/cus-samples.html>

S6

VMC Protocol specification

VirtualMotionCaptureProtocol

<https://protocol.vmc.info/english.html>

S7

OBS Remote Control Guide

OBS Project

<https://obsproject.com/kb/remote-control-guide>

L1

Project manifest and lock file

Local project

[Packages/manifest.json](#), [Packages/packages-lock.json](#)

L2

Installed extOSC package manifest

Local package cache

[Library/PackageCache/com.iam1337.extosc@b7c2bfa81633/package.json](#)

L3

Installed extOSC examples

Local package cache

[Examples~/01-13](#)

L4

Installed extOSC runtime source

Local package cache

[OSCReceiver.cs](#), [OSCTransmitter.cs](#), [OSCMessages.cs](#), [OSCValue.cs](#), [OSCUilities.cs](#)

L5

Installed extOSC editor/UI source and icons

Local package cache

[OSCMenueOptions.cs](#), [Resources/extOSC/*.png](#)

L6

Installed URP runtime source

Local package cache

[UniversalRenderPipelineAsset.cs](#), [Bloom.cs](#), [VolumeProfile.cs](#)